

WAFT PROJECT

WAFT SELF-STUDY SYSTEM IMPLEMENTATION

Self-Monitoring and Iterative Study Framework

DOC-SELF-STUDY-20260113

FEATURE:	Self-Study System
DATE:	2026-01-13
STATUS:	COMPLETE

 INFO

MILESTONE: WAFT can now monitor and study itself iteratively

IMPLEMENTATION DOCUMENTATION

WAFT SELF-STUDY SYSTEM IMPLEMENTATION

Date: January 13, 2026

Version: 1.0.0

Status: Complete and Operational

EXECUTIVE SUMMARY

This document describes the implementation of WAFT's self-study system, which enables the framework to monitor and iteratively study itself through evidence-based proof methodology. The system provides comprehensive self-assessment capabilities across multiple dimensions, tracks changes over time, and generates professional documentation.

1. OVERVIEW

1.1 Purpose

The WAFT Self-Study System enables:

- **Self-Monitoring:** WAFT can verify its own health and state
- **Iterative Tracking:** Compare studies over time to identify trends
- **Evidence-Based Verification:** Uses rigorous proof methodology
- **Multi-Dimensional Analysis:** Studies templates, code, system health, work efforts, and epistemic state
- **Comprehensive Documentation:** Generates PDF case files with full evidence

1.2 Architecture

The system is built on three core components:

1. **WAFTSelfStudy Class:** Main orchestrator that manages study execution
2. **ProofCaseBuilder Integration:** Reuses existing proof system for verification
3. **Study Areas:** Five distinct areas of self-assessment

2. CORE COMPONENTS

2.1 WAFTSelfStudy Class

Location: scripts/waft_self_study.py

The main class provides:

- Study area definition and management
- Study execution orchestration
- History tracking and comparison
- Health score calculation

Key Methods:

- `get_study_areas()`: Returns all available study areas with claims
- `run_study()`: Executes study on specified area(s)
- `_run_area_specific_checks()`: Runs area-specific assumption validation
- `_calculate_overall_health()`: Computes system health score
- `get_study_history()`: Retrieves recent study results
- `compare_studies()`: Compares two studies to identify changes

2.2 Study Areas

Five distinct study areas are defined:

2.2.1 Template Health

- Claim: "All PDF templates are properly formatted and free of problematic styling"
- Method: Template scanning for black bars, CSS issues, formatting problems
- Evidence: File paths, line numbers, code snippets of verified templates

2.2.2 Code Quality

- Claim: "Source code follows project standards and best practices"
- Method: Pattern matching for imports, error handling, type hints
- Evidence: Code analysis results with file-level attribution

2.2.3 System Health

- Claim: "WAFT system is healthy and operational"
- Method: File existence checks, git status, dependency verification
- Evidence: System state snapshots, file checks, git status

2.2.4 Work Efforts

- Claim: "Work efforts system is properly structured and maintained"
- Method: Directory structure validation, index file checks
- Evidence: Structure analysis, index file counts, organization compliance

2.2.5 Epistemic State

- Claim: "Epistemic tracking is functioning correctly"
- Method: Empirica integration checks, initialization verification
- Evidence: Empirica status, initialization state, integration health

3. WORKFLOW

3.1 Study Execution Flow

1. Area Selection: User specifies area(s) or selects all
2. Proof Builder Initialization: Creates ProofCaseBuilder with area claim
3. Verification: Runs standard verification checks (date, disk, git, etc.)
4. Assumption Validation: Executes area-specific checks
5. Verdict Determination: Calculates PROVEN/DISPROVEN/INCONCLUSIVE
6. Case File Generation: Builds comprehensive markdown case brief
7. PDF Creation: Generates professional PDF binder
8. Results Storage: Saves JSON results for history tracking

3.2 Evidence Collection

Each study collects evidence including:

- Source Files: Paths relative to project root
- Line Numbers: Specific locations of findings
- Code Snippets: Relevant code sections
- Verification Methods: How evidence was collected
- Confidence Scores: 0.0-1.0 confidence levels

3.3 Health Scoring

Overall health is calculated as:

- Average Confidence: Mean confidence across all studies (60% weight)
- Proven Ratio: Percentage of studies with PROVEN verdict (40% weight)

Health Status Levels:

- Excellent: ≥ 90
- Good: ≥ 75
- Fair: ≥ 50
- Poor: < 50

4. INTEGRATION POINTS

4.1 Proof System Integration

The self-study system leverages the existing proof system:

- Uses ProofCaseBuilder for verification and case file generation
- Reuses verification checks (date, disk, git, templates)
- Extends assumption validation with area-specific checks
- Generates PDF binders using existing brief system

4.2 WAFT Status Integration

Complements /waft-status command by:

- Adding evidence-based verification to status checks
- Providing historical tracking capabilities
- Enabling trend analysis over time
- Generating comprehensive documentation

4.3 Work Efforts Integration

Studies the work efforts system itself:

- Verifies directory structure
- Checks index file presence
- Validates Johnny Decimal organization
- Ensures system integrity

5. USAGE EXAMPLES

5.1 Daily Health Check

```
python scripts/waft_self_study.py --area system_health
```

Quick verification of system health status.

5.2 Weekly Comprehensive Review

```
python scripts/waft_self_study.py --all
```

Full system study across all five areas.

5.3 Monthly Trend Analysis

```
python scripts/waft_self_study.py --history 30  
python scripts/waft_self_study.py --compare study_results_20260101.json study_results_20260131.
```

Track changes and identify trends over time.

5.4 Template Health Verification

```
python scripts/waft_self_study.py --area template_health
```

Verify all templates are properly formatted (prevents black bar regressions).

6. OUTPUT FILES

6.1 Case Files

Location: `_work_efforts/self_studies/study_{area}_{timestamp}.md`

Format: Comprehensive case brief with:

- Abstract with evidence summary
- Hypothesis (H_0 and H_1)
- Methodology explanation
- Verification evidence
- Assumption validation
- Analysis section
- Conclusion & Summary
- Appendix with metadata

6.2 PDF Binders

Location: `_work_efforts/self_studies/STUDY_{AREA}_{timestamp}.pdf`

Format: Professional PDF with:

- Cover page with verdict
- Full case file content
- Evidence with sources
- Confidence scores

6.3 Results JSON

Location: `_work_efforts/self_studies/study_results_{timestamp}.json`

Format: Structured data for:

- History tracking
- Comparison analysis
- Trend detection
- Health score calculation

7. IMPLEMENTATION DETAILS

7.1 File Structure

```
scripts/
  waft_self_study.py          # Main self-study system
  prove_it_comprehensive.py   # Proof system (reused)

.cursor/commands/
  waft-self-study.md          # Command documentation

_work_efforts/self_studies/
  study_{area}_{timestamp}.md  # Case files
  STUDY_{AREA}_{timestamp}.pdf # PDF binders
  study_results_{timestamp}.json # Results JSON
```

7.2 Dependencies

- ProofCaseBuilder from scripts/prove_it_comprehensive.py
- BriefDocument from src/waft/brief.py
- Standard library: json, pathlib, datetime, subprocess, re

7.3 Area-Specific Checks

Each study area implements custom assumption validation:

- Template Health: Reuses template scanning from proof system
- Code Quality: Pattern matching for imports, error handling, type hints
- System Health: File existence, git status, dependency checks
- Work Efforts: Directory structure, index file validation
- Epistemic State: Empirica initialization and integration checks

8. FUTURE ENHANCEMENTS

8.1 Planned Features

- Automated Scheduling: Run studies on schedule (daily, weekly)
- Regression Detection: Automatic alerts when health degrades
- Trend Visualization: Charts and graphs of health over time
- Custom Study Areas: Allow users to define new study areas
- Integration with CI/CD: Run studies as part of build pipeline
- Comparative Analysis: Compare against baseline studies
- Recommendations: Suggest improvements based on study results

8.2 Extensibility

The system is designed for easy extension:

- Add new study areas in `get_study_areas()`
- Implement area-specific checks in `_run_area_specific_checks()`
- Extend health calculation in `_calculate_overall_health()`

9. BEST PRACTICES

9.1 Regular Studies

- Run weekly comprehensive studies (`--all`)
- Perform daily health checks before major work
- Study relevant areas before refactoring

9.2 Trend Monitoring

- Compare studies monthly to track trends
- Review health scores over time
- Identify regressions early

9.3 Documentation Review

- Review PDF binders for detailed evidence
- Check case files for specific findings
- Use JSON results for automated analysis

10. CONCLUSION

The WAFT Self-Study System provides a comprehensive framework for self-monitoring and iterative study. By leveraging the existing proof system methodology, it enables rigorous verification across multiple dimensions while maintaining consistency with WAFT's evidence-based approach.

The system is operational and ready for use. Regular studies will help maintain system health, identify regressions early, and provide comprehensive documentation of WAFT's state over time.

11. APPENDIX

11.1 Command Reference

```
# Study all areas
python scripts/waft_self_study.py --all

# Study specific area
python scripts/waft_self_study.py --area template_health

# View history
python scripts/waft_self_study.py --history 10

# Compare studies
python scripts/waft_self_study.py --compare file1.json file2.json
```

11.2 Study Areas Reference

Area	Claim	Key Checks
template_health	All templates properly formatted	Black bars, CSS, formatting
code_quality	Code follows standards	Imports, error handling, type hints
system_health	System is healthy	Files, git, dependencies
work_efforts	Work efforts structured	Directory, indexes, organization
epistemic_state	Epistemic tracking works	Empirica, initialization

11.3 Health Score Formula

$$\text{health_score} = (\text{avg_confidence} * 0.6 + \text{proven_ratio} * 0.4) * 100$$

Where:

- avg_confidence: Mean confidence across all studies
- proven_ratio: Percentage of studies with PROVEN verdict

11.4 File Locations

- Script: scripts/waft_self_study.py
- Documentation: .cursor/commands/waft-self-study.md
- Output: _work_efforts/self_studies/